Capstone 2 Project Update / Report

Group Members:

- **Dinglasa, Danny Jr. D.P.**
- **Jambre, Benedict C.**
- **Lutian, Erzhad Dominic S.**

Project Title: VoltWise: An IoT Non-Intrusive Load Monitoring (NILM) System for Smart Home Energy Monitoring and Automated Protection

Project Description:

VoltWise is an IoT and Machine Learning–powered smart home energy monitoring and automated protection system based on Non–Intrusive Load Monitoring (NILM) principles. It tracks real-time electrical consumption, detects anomalies and vampire loads, forecasts utility costs, and triggers automated physical safety shutoffs to protect appliances from electrical hazards.

AI/ML Model, Algorithm, IOT or Other Key Technology Used/Implemented:

The **Voltwise** is a single point of measurement where it measures all of the appliances in one point, the voltwise is tend to disaggregate the electrical signal to identity which appliances is being monitored/controlled, to utilize this **voltwise** is using **k-mean clustering** which clusters all the electrical signal to identify which appliances is connected to the IoT device, The main sensor used is the **PZEM-004T** and **CT-CLAMP**which is responsible to monitor the electrical fingerprint such as **Voltage, Wattage, Power Factor, Current** and **Frequency** and one other key IoT hardware is the **30A relay module** this module is responsible for the power control of the IoT device.

Current Completion Percentage (%):

Components	Percentage with Comments
- **ML Model Completeness**	- **10% (Still Collecting Data to Train Model)**
- **Application Software Completeness**	- **90% (More UI polishing and Word of choice)**
- **Backend Integration Completeness**	- **80% (API endpoints and functions are intact, Model Integration still missing inaction since it is still not completed yet)**

- **IoT Completeness**	- **90% (Polishment of IoT portal, and Adding one more socket)**

Project Status:

1. Already Working (Implemented & Operational)

Layer	Feature	Details
Mobile App	Live Telemetry Dashboard (dashboard.tsx)	Real-time active power (kW), energy (kWh), voltage and current via WebSockets (WSS MQTT) from HiveMQ Cloud
Mobile App	Interactive Range Navigation (RangeNavigator.tsx)	Day, Week, Month and Billing Cycle power charts, with 3-hour label thinning and a custom month picker
Mobile App	Virtual Device Management (devices.tsx)	CRUD for appliances/rooms, status toggles, device photo uploads
Mobile App	Cost Estimation & Analytics (analytics.tsx)	Projected monthly bill from user-set tariffs, appliance consumption rankings, SVG category donut charts
Mobile App	Alert Feed & Anomaly Review (alerts.tsx)	Critical/Warning/Info alerts, read/unread tracking, modal detail view
Mobile App	Security & Preferences	Biometric/PIN App Lock (AppLockContext.tsx), Dark/Light theme, local storage reset
Mobile App	Appliance Data Collection Suite (data-collection.tsx)	Samples real appliance telemetry and exports labeled CSV datasets for ML training
Mobile App	Sensor Pairing & Provisioning UI (iot-settings.tsx, iot-provision.tsx)	Step-by-step wizard to scan Wi-Fi and send credentials and Device UID to the ESP32
Mobile App	Demo Mode (DemoFab.tsx)	In-memory sample household telemetry overlay, no server mutations
Backend API	Authentication & Revocable Sessions	bcrypt password hashing, JWT with revocable session IDs (sid), multi-device session revocation

Backend API	Modular Data Endpoints	Route/controller/service modules for /api/dashboard, /api/devices, /api/alerts, /api/analytics
Backend API	Data Export Engine	CSV streaming with UTF-8 BOM and formula-injection sanitization, plus structured JSON downloads
Backend API	HiveMQ MQTT Ingestion Bridge (mqtt.ts)	Ingests 2-second telemetry packets into PostgreSQL (EnergyReading), manages hardware relay commands
Backend API	Heuristic Step-Change Load Detection (loadDetector.ts)	EMA baseline step-change detector, auto-alerts when an unmapped appliance powers on
Backend API	Relational Persistence (schema.prisma)	PostgreSQL with Prisma 7 ORM, migrations, seeded test data
IoT Hardware	Sensor Telemetry Acquisition (main.cpp)	PZEM-004T v3.0 readings every 2 s: voltage, current, active power, energy, frequency, power factor
IoT Hardware	Dual-Tier Relay Switching	Independent control of Low Load (10A) and High Load (30A) circuits (e.g., aircon, water heater)
IoT Hardware	Firmware-Level Safety Cutoff	Emergency cutoff above MAX_ALLOWED_POWER_WATTS, with a 30-second auto-shutdown countdown
IoT Hardware	Captive Portal Wi-Fi Provisioning (provisioning_server.h)	ESP32 AP web server, NVS flash storage for network config
IoT Hardware	Encrypted MQTT Communication	MQTTS over TLS (port 8883), Last Will and Testament (LWT) online/offline presence
ML Engine	30-Second Window Feature Engineering (feature_engineering.py)	10 statistical features per window (mean voltage/current, power_std, inrush peak power_max, power factor)
ML Engine	K-Means Model Pipeline (train.py)	K = 6, Silhouette Score 0.6377; separates aircon continuous vs. inrush surge and fridge cooling vs. idle
ML Engine	Euclidean Distance Anomaly Thresholding	95th percentile outlier detection (> 1.7950) flags irregular electrical signatures

ML Engine	FastAPI Inference Server (main.py)	Standalone endpoints: /health, /clusters, /telemetry, /predict

2. Currently Under Development (In Progress / Partially Integrated)

Item	Current State	What's Missing
Backend-to-ML Service Bridge	FastAPI ML service works on its own	Express backend doesn't call it yet; bill predictions and analytics still use SQL aggregations and heuristic rules
End-to-End Automated Anomaly Intervention Loop	Firmware cutoff triggers on raw wattage overloads	Automatic shutdown on ML-detected anomalies (e.g., compressor stall, degradation) isn't connected end-to-end
Sub-circuit Relay Granularity vs. Virtual Devices	App has a virtual registry of 6+ appliances	Hardware uses a dual-tier master relay (Low-Load and High-Load buses); mapping virtual devices to discrete contactors is in progress
Mobile Hotspot Handoff Optimization	Captive portal provisioning logic is in place	OS-level Wi-Fi handoff (ESP32 setup AP to home Wi-Fi) is still being refined across Android and iOS versions

3. Not Yet Implemented (Planned / Future Roadmap)

Feature	Current Limitation
Real-Time Composite NILM Disaggregation	ML only profiles single-appliance datasets; it can't unmix simultaneous multi-appliance loads from one aggregate signal
Remote Push Notifications (APNs / FCM)	Alerts only appear in-app while the app is foregrounded or active; nothing is sent when the app is fully terminated
Advanced Dynamic Tariff Schedules	Flat rate only (default 10.5 ₱/kWh); no tiered, Time-of-Use or seasonal pricing
Time-Series DB Downsampling (TimescaleDB / Timestream)	Telemetry goes straight into PostgreSQL; no retention policies, continuous rollups (2-second to 1-minute/1-hour), or cold storage

Data/Dataset Details:

Attribute	Details

Data Source	Physical PZEM-004T v3.0 sensor connected to an ESP32, collected live via the VoltWise App Data Collector over MQTT
Data Type	High-frequency, multivariate time-series AC electrical telemetry (raw) and 30-second statistical window vectors (processed)
Total Raw Samples	912 raw telemetry readings (~30 minutes of real appliance operation)
Total Processed Windows	62 feature-engineered windows (10 electrical summary statistics per window)

1. Data Source & Collection Method

Component	Details
Sensing Hardware	PZEM-004T v3.0 digital multi-function power/energy meter measuring mains AC line voltage and current transformer (CT) coil readings
Microcontroller	ESP32 querying the sensor via hardware UART (Serial2 on GPIO 16 RX / GPIO 17 TX) using the Modbus-RTU protocol at 9600 baud
Sampling Interval	Once every 2.0 seconds (about 0.5 Hz)
Ingestion & Recording Pipeline	Readings sent over TLS-encrypted MQTT (HiveMQ Cloud) to the VoltWise app; the built-in Data Collection Suite captured, labeled, and exported sessions to timestamped CSV files

2. Data Type & Schema

The data exists in two tiers: raw sensor telemetry and feature-engineered windows.

A. Raw Telemetry Schema

Each record is an instantaneous snapshot of the electrical circuit with 8 parameters.

#	Field	Description	Range / Values
1	timestamp	ISO-8601 UTC timestamp	YYYY-MM-DDTHH:MM:SS.mmmZ
2	appliance	Ground-truth label of the monitored device	"Air Conditioner" or "Refrigerator"
3	voltage_v	Line RMS voltage (V)	229.3 V to 239.3 V
4	current_a	RMS current (A)	0.000 A idle up to 19.189 A inrush

5	power_watts	Active real power (W)	1.0 W up to 3,665.5 W peak
6	energy_kwh	Cumulative active energy from the PZEM register	Cumulative
7	frequency_hz	AC grid frequency (Hz)	59.8 Hz to 60.3 Hz
8	power_factor	Active-to-apparent power ratio	0.30 to 1.00

B. Processed Feature Schema (data/processed/windows.csv)

To eliminate single-reading transient noise, feature_engineering.py groups the 2-second readings into non-overlapping 30-second windows (15 readings per window) and extracts 10 statistical features.

Feature Group	Features
Voltage	voltage_mean
Current	current_mean, current_max
Power	power_mean, power_max, power_min, power_std (power volatility)
Energy	energy_delta_wh (energy consumed within the 30-second interval)
Grid / Efficiency	frequency_mean, power_factor_mean

3. Data Quantity & File Breakdown

Raw Datasets: As For today this is currently the Dataset: [Dataset Link]

Dataset File	Target Appliance	Row Count	File Size	Duration Covered
voltwise_air_conditioner_2026-09-25T23-45-16.csv	Air Conditioner	458 readings	37.6 KB	~15.3 minutes
voltwise_refrigerator_2026-09-26T00-04-25.csv	Refrigerator	454 readings	35.2 KB	~15.1 minutes
Total Raw Telemetry	n/a	**912 readings**	**72.8 KB**	**~30.4 minutes**

Processed Training Windows (windows.csv)

Attribute	Details

Total Windows	62
Air Conditioner Windows	30
Refrigerator Windows	32
Window Size	30 seconds (15 samples per window)
Dataset Shape	62 × 10 feature matrix

4. Physical Operational States Captured

The dataset captures 6 distinct operational behaviors, profiled in cluster_profiles.csv.

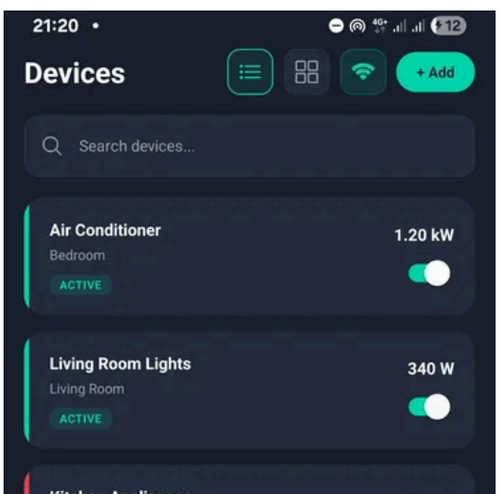
Cluster	Samples	Mean Power	Mean Current	Power Factor	Operational Interpretation
0	29 windows	639.5 W	3.13 A	0.86	Air Conditioner: normal continuous running state
1	16 windows	24.3 W	0.09 A	0.99	Refrigerator: low-load / compressor cycling
2	1 window	505.2 W	2.64 A	0.93	Air Conditioner: motor inrush spike (3,665.5 W / 19.2 A peak)
3	6 windows	13.0 W	0.05 A	1.00	Refrigerator: standby / idle mode
4	5 windows	315.3 W	1.54 A	0.93	Refrigerator: active cooling cycle
5	5 windows	12.5 W	0.06 A	0.75	Refrigerator: low-PF standby state

Screenshot(s) or Image(s) of the Updated Project:

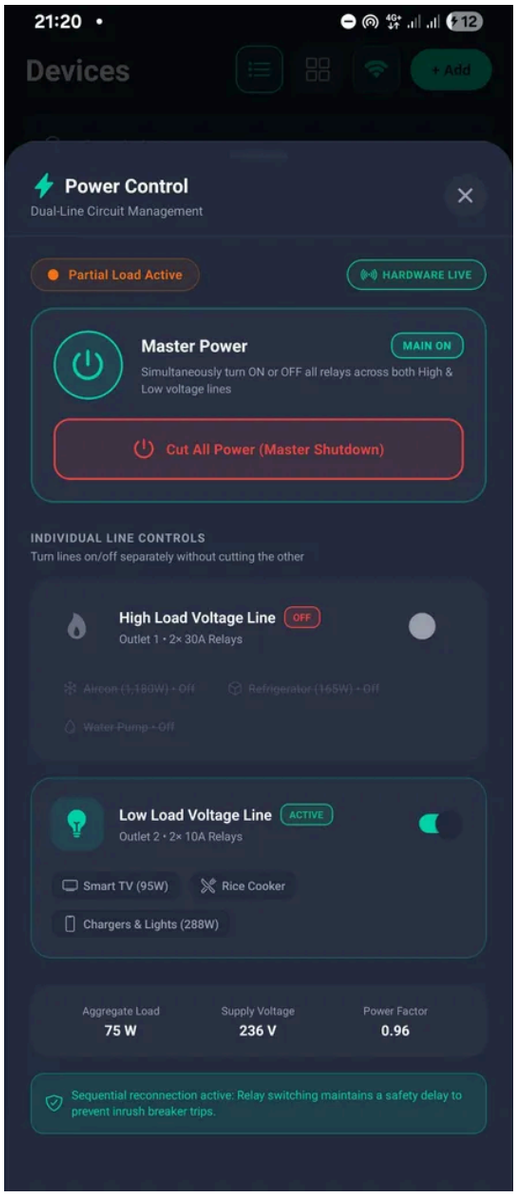
Dashboard Screen



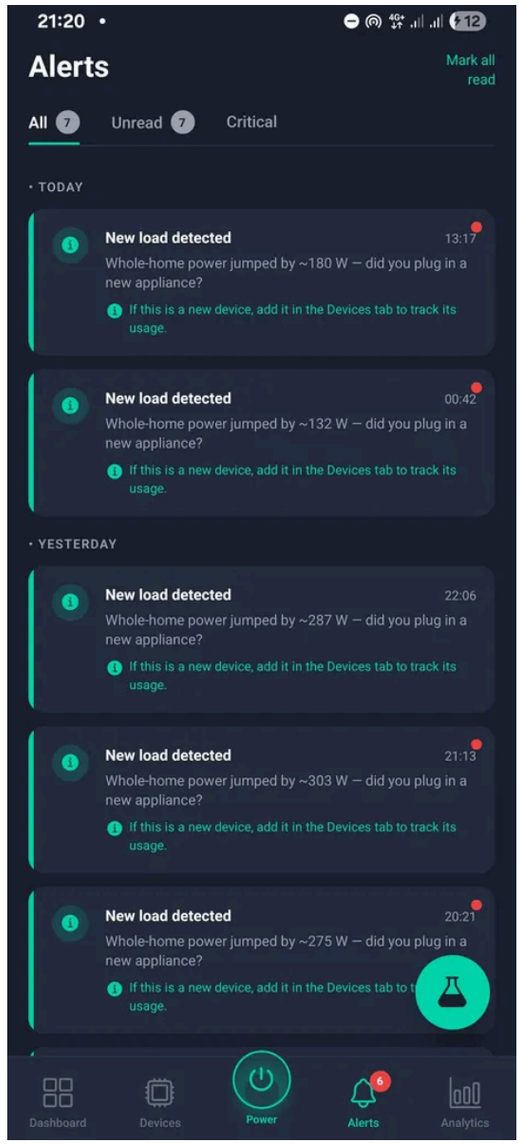
Devices Screen



Power Control Modal



Alert Screen



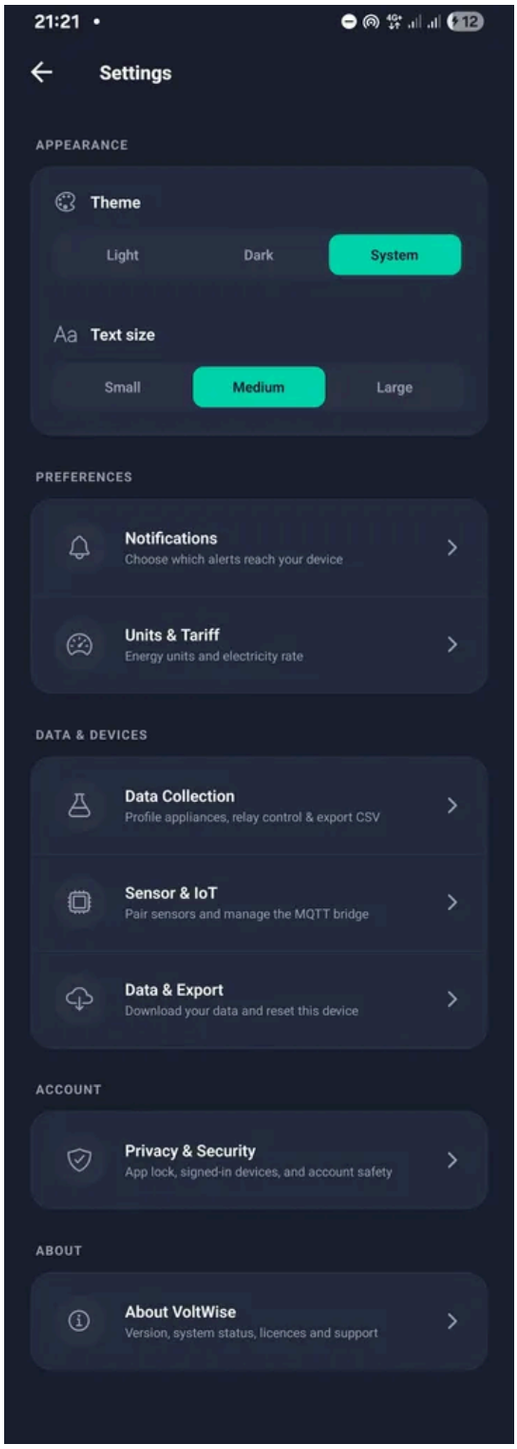
Analytics Screen



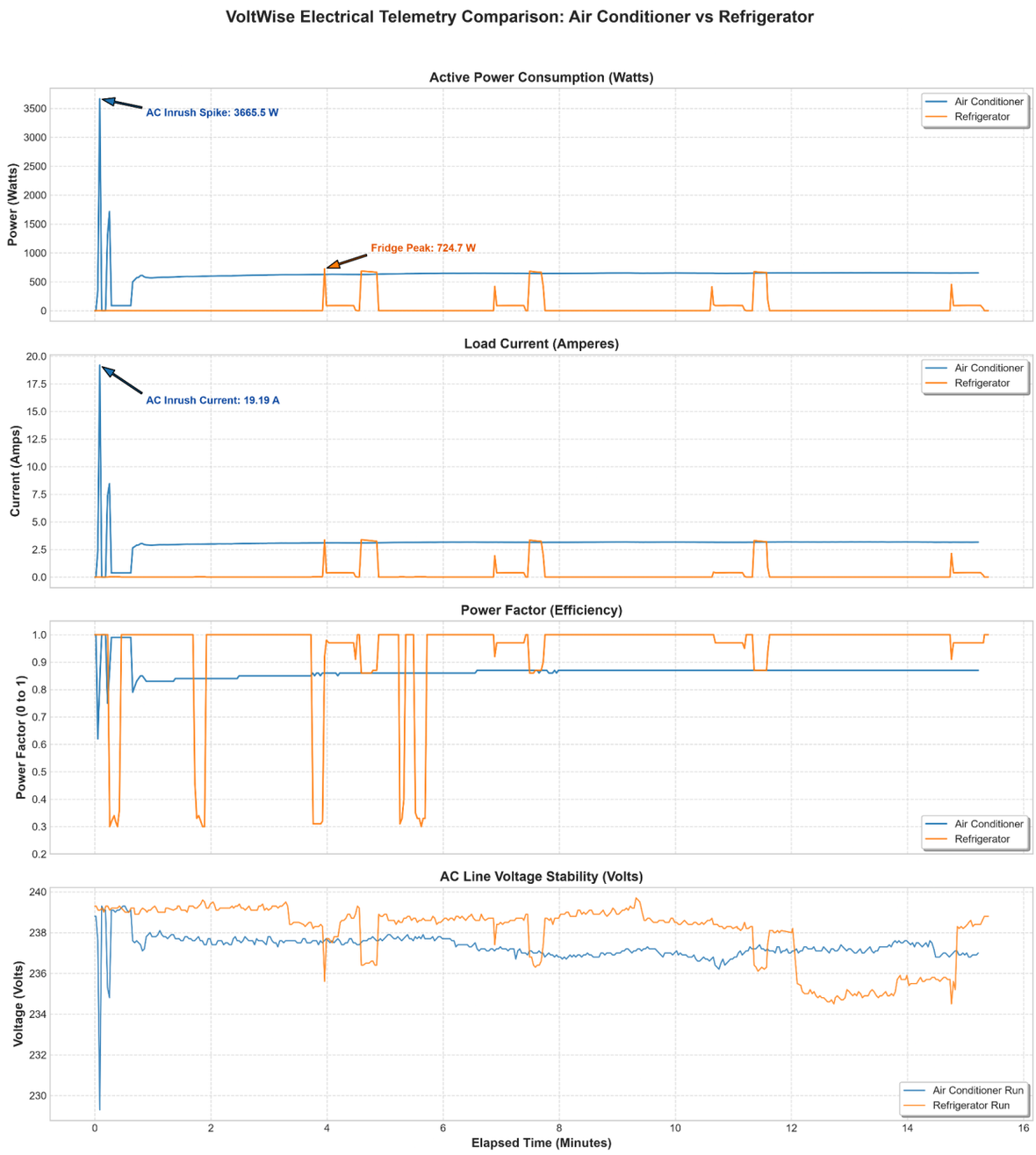
Notification Bubble



Settings Screen



Dataset Graphs: Air Conditioner vs Refrigerator Electrical Signal Comparison



IoT Device – It has a custom made power rail system that is Powered By AC Voltage but transformed using Hi-Link Transformer 240V AC to 5v DC, With ESP32, 30A Relay Module, and PZEM-004T and Clamp

